

	A	B	C	D	E	F	G	H	I	J	K	L
1	Timestamp	Voltage	Current_mA	TotalPower	Frequency	PowerFactor	Smin_Pred	Daily_Pred	Yearly_Pred	Daily_Peak	Monthly_Peak	Yearly_Peak
2	2026-04-02	29.9	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
3	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
4	2026-04-02	29.7	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
5	2026-04-02	29.7	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
6	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
7	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
8	2026-04-02	29.8	69	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
9	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
10	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
11	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
12	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
13	2026-04-02	29.8	68	2100	50	1	2.0999999	2.1	804.825	2100	2100	2100
14	2026-04-02	30	0	0	50	0	2.03154469	2.0712329	789.8	2100	2100	2100
15	2026-04-02	30	0	0	50	0	1.969645262	2.0432432	783.07997	2100	2100	2100
16	2026-04-02	30.1	0	0	50	0	0.96779639	1.913921	733.51139	2100	2100	2100

Fig. 8: Microsoft Excel worksheet

100MVA) are monitored using suitably rated CTs and PTs to provide safe isolation and accurate measurement.

Software and code

The system uses multiple software tools for data acquisition, cloud storage, AI-based prediction, and data visualisation. Arduino IDE is used to develop and upload the ESP32 firmware for sensor data acquisition. Firebase real-time database stores the measured parameters in the cloud. Python processes the stored data and performs AI-based load prediction using libraries such as TensorFlow or Scikit-learn. Finally, Microsoft Excel stores and visualises the measured and predicted data.

Sensor code

The sensor code is developed in the Arduino IDE. It reads voltage, current, power, energy, frequency, and power factor from the PZEM-004T module. Before uploading the code, configure the Wi-Fi SSID,

password, Firebase API key, and real-time database URL. Next, select the appropriate ESP32 board in the Arduino IDE and upload the program.

Once the ESP32 connects to the Wi-Fi network, it continuously reads data from the PZEM-004T module and uploads the measurements to the Firebase real-time database in real-time.

After the sensor data becomes available in Firebase, a Python program retrieves the stored measurements and applies an AI model, such as LSTM, to predict future electrical load. The program also calculates peak demand and exports both the measured and predicted values to an Excel spreadsheet for visualisation and further analysis. The Python implementation for AI-based load prediction is described in the next section.

LSTM (long short-term memory) is a deep learning architecture used for forecasting time-series data such as electrical load, weather, stock prices, and energy consumption.

Configuration of Firebase

Firebase provides cloud data storage and analytics. Configure it using the following steps.

First step: Create a Firebase project

1. Open the Firebase console: <https://console.firebase.google.com>
2. Click add Project
3. Enter a project name and click Continue
4. Disable Google Analytics (optional), then click Create Project
5. Wait for the project to be created and click Continue

Second step: Add a web app

1. In Project Overview, click Add app and select the web (</>) icon
2. Enter an app name (for example, esp32camsecurity)
3. Click Register app
4. Firebase will display the project configuration, including the web API key, project ID, and other details. Save the web API key for use in the ESP32 code

Third step: Enable Firebase realtime database

1. In the left panel, go to Build→